Career Description · Jinhee Mok

jinheemok815@gmail.com · Portfolio https://jin-tonix.github.io · GitHub https://github.com/Jin-tonix

I majored in law and worked as a daycare operations manager and an English teacher. In 2024 I learned software development at Himedia Academy. As an intern at We:Review, I automated receipt review with OCR and an LLM. Now I move repetitive field work into AI agents and automation. I design systems that stop at candidates and suggestions, and staff make the final decision. I hand much of the coding to an AI coding tool (Claude Code). I decide what to build and what to turn on in production, and I check the results.

StyleSeller (SNS Commerce platform) · PL (AI and Automation)

2025.11 to present

I moved work that staff repeated by hand, such as organizing counterpart records, answering inquiries, and calculations, into AI agents and an internal work tool. I also handle deployment and operations.

1. Status Board · Operations Ledger (built 2026.10.19 to 10.23, in production)

- **Role**: Designed the ledger schema and the counterpart linking rules. Built the intake API, the mail collector, and 6 screen tabs. Ran DB tests and verified the production screens.
- **What I built**: It summarizes KakaoTalk rooms, the order@ mailbox, Slack, and Excel order sheets attached to emails by day, and puts them in date order on each counterpart's calendar. If the same record arrives more than once, the ledger keeps it only once. Records are linked to counterparts without an LLM, by a fixed rule order (thread, business registration number, KakaoTalk room, email, and so on). If the keys conflict, the record stays unclassified and staff decide.
- **Results**
  - Deal stages of 352 counterparts (147 brands, 135 sales partners, 70 sellers) on one screen (2026-10-24)
  - Collects about 100 to 200 records per weekday. Staff link the 421 records the system could not link automatically on the counterpart check screen
  - Rewrites summaries for up to 300 counterparts every day at 18:05, and stops that day's batch if the daily cost passes KRW 2,000
- **Tech**: Next.js, React, Tailwind, Vitest, Python 3.12, Pydantic v2, Postgres RLS, Gemini Flash family (via LiteLLM)

2. Vendor CS Agent (2026.06 to present)

- **Role**: Designed the collection path, the vendor gate, and the evidence check rules. Built the server agent, sending and delivery confirmation in the kakaocli fork, and the Windows collector. Handle incidents and operations.
- **What I built**: KakaoTalk has no official API, so staff PCs collect the conversations and upload them to the server, and replies are sent from staff Macs. The agent checks the room name against the list of vendors it serves and does not answer if it cannot identify the vendor. For tagged questions, it searches the vendor's records and the knowledge hub to answer. Code checks the amounts, percentages, and brands in the draft against the evidence. If they do not match, the agent searches again and rewrites the draft.
- **Results**
  - Fixed a resend problem where one message came in 51 times, with 15-second deduplication
  - Fixed a macOS Accessibility freeze that made 31 of 147 sends (21%) fail, with a 3-second timeout and delivery confirmation (comparing message counts in the KakaoTalk DB)
  - 600 commits and 107 test files in the vendor agent (2026-09-23)
- **Tech**: kakaocli fork (Swift), Windows read-only memory collector (Python), Python 3.12, LightRAG, Gemini Flash family (via LiteLLM), Google Sheets and Drive

3. Internal Work Tool · Pricing, Matching, Personalized KakaoTalk (late 2026.03 to present)

- **Role**: Designed the price calculation structure, the matching candidate rules, and the send approval criteria. Built and run the pricing code, catalog extraction, matching, send automation, and staff PC workers. The seller recommendation logic is owned by a colleague in charge of CRM.
- **What I built**: I moved the Google Sheets price formulas into code, down to the rounding units. The LLM only returns the input values to change as JSON, and code recalculates the amounts. The LLM extracts product candidates from catalogs (PDF, image, Excel, URL), and only items that staff confirm are registered. Personalized KakaoTalk messages go to the recipients received from the CRM. Sends to 500 or more people wait in an approval queue.
- **Results**
  - 43 price calculation test cases, some compared field by field with the sheet values of real products
  - Personalized KakaoTalk sent to 261 people in one day (2026-09-22)
  - Seller search workers run on 9 staff PCs, and the shared key was replaced with a key per PC (2026-09)
- **Tech**: Next.js, React, Vitest, Postgres+PostgREST, Gemini Flash family (via LiteLLM), knowledge hub RAG search, kakaocli

4. LightRAG Knowledge Hub · Email Agent (2026.03.31 to present)

- **Role**: Designed the workspace registry, the search planning and evidence judging steps, and the draft verification order. Built the email-agent server, the checker, and the sent-mail learning cursor. Handle deployment and incident recovery.
- **What I built**: A knowledge hub shared by Vendor CS, Influencer CS, Spark, and sales partner matching. For inquiries to the shared mailboxes, it puts reply drafts with evidence into the Gmail drafts folder of the person in charge, and staff send them. It checks up to 6 claims in a draft against the evidence. Claims without evidence are rewritten, and if unsupported claims remain after the rewrite, the draft is dropped.

- **Results**
  - 4 agents share 13 workspaces, and the server does not start if a workspace name does not match
  - Found the cause of the mail draft edit rate rising from 37% to 82% (a missing rerank step) in the commit history, and restored it on 2026-07-23
  - Fixed 48 of 1,943 sent emails being left out of learning, by changing how the cursor is saved
- **Tech**: Python 3.12, FastAPI, Pydantic v2, LightRAG (Postgres/pgvector + Neo4j), FlashRank, Gemini Flash family (via LiteLLM), Gmail, pytest

5. Agent Platform · Spark (2026.05 to present)

- **Role**: Designed the work order, delegation guard, and tool gate structure. Built Spark SQL, one-time approval, the LiteLLM cost cap, and the Jev connection. Wrote the evaluation set and deployed to the VM.
- **What I built**: An internal agent platform forked from Paperclip. Staff use Spark to ask questions of 4 business DBs with read-only access, and hand work to the CEO agent. Delegation depth is limited to 2, and each work order allows 5 delegations. Tool approval is enforced by the server and the DB, not by prompts. Jev only offers shared skill candidates as hints, and the agent doing the work makes the choice.
- **Results**
  - 109 shared skills. Jev candidate recall is 85.5% at top 1 and 91.3% in the top 3 (69 questions with known answers, 2026-10-24)
  - Each approval runs only once (10 concurrent requests, 1 execution, confirmed by tests)
  - Paid calls are capped at KRW 30,000 per day. Above the cap, the server responds with 403 so callers do not retry
- **Tech**: Paperclip fork (Express 5, Drizzle, PostgreSQL), MCP SDK, Hermes, Python 3.12, FastAPI, LiteLLM, Redis, Langfuse, Gemini Flash family (via LiteLLM), Cloudflare Workers AI

6. Content Autopilot (2026.10.03 to present)

- **Role**: Designed the stage state machine and the publishing defaults. Built the orchestrator, the publish gate and DB slots, and the QA gate. Ran the production canary and restart scenario tests.
- **What I built**: When staff request a Reel in the work tool, it writes the script, renders a 9:16 MP4, and gets approval on a preview. A state machine without an LLM moves the stages forward. The defaults are manual publishing, dry-run for recurring automation, and paid generation off. The "AI-generated content" label is added automatically.
- **Results**
  - The production canary verified the flow up to saving a 1080×1920 MP4 to Drive (2026-10). 0 real SNS posts (publishing is simulated)
  - One post per account per day, enforced by a DB index
  - About 300 test files across 3 services
- **Tech**: content-orchestrator, Hermes, Gemini Flash family (via LiteLLM), video-studio, HyperFrames, ffprobe, Google Drive, Supabase (PostgreSQL)

7. Staff PC Deployment · Operations (2026.02 to present)

- **Role**: Designed the install, authentication, and update structure and the server-side reference date. Built the installer, workers, the Windows collector, and changes to the kakaocli fork. Handle deployment, incidents, and monitoring cron jobs.
- **What I built**: KakaoTalk and Instagram tasks run on staff Macs and Windows PCs. Install is a one-line command. Authentication uses a per-device key issued with a one-time token. Updates run on an automatic schedule, with a different time for each PC. The server deploys only from committed images and stops the deploy if an image is missing.
- **Results**
  - One-line install deployed to 11 staff PCs (Mac and Windows) (as registered on 2026-09-16)
  - On 2026-08-20, a cloud DB usage limit was exceeded and all work tool APIs stopped. I set up the same API (PostgREST) on a VM and recovered the same day, with about 100 DB call sites in the app left unchanged
  - Moved 4 DBs to self-hosting (2026-08-06 to 08-25)
- **Tech**: macOS bash and Windows PowerShell installers, launchd and Task Scheduler, Node workers, Python memory collector, VM Postgres+PostgREST+nginx, SSE push, Docker, Slack monitoring cron

We:Review (Welead) · AI Developer Intern

2025.08 to 2025.10

- Built a review pipeline that extracts products and prices from receipt images as JSON: PaddleOCR tile splitting, DBSCAN line reconstruction, few-shot prompting with an open LLM (Qwen family), served with FastAPI.
- Raised review accuracy from 70% to 95% and cut review time by 85% (my own measurement).

Previous Experience

- Montekids English Academy · Btop Kinder · Jeongsang Language School · English Teacher (2018.08 to 2024.02): lesson design, teaching materials, parent consultations.
- Jeonwon Daycare Center and After-School Program · Operations Manager (2015.03 to 2018.02): teacher hiring, curriculum and event planning, running the center on behalf of the director.